Secure Push Gateway

Complete Technical Documentation

Automated Code Security Scanning Platform

Spring Boot 3.3.0 | React 18 | JavaParser AST | MySQL

Author	Ashish
Version	1.0.0
Java	21 (LTS)
Spring Boot	3.3.0
Frontend	React 18 + Vite 5
Database	MySQL 8.0+

Table of Contents

1. Introduction & Project Overview

2. System Architecture

3. Technology Stack & Software Used

4. Concepts & Design Patterns

5. Backend Deep Dive

- **5.1** Project Structure
- **5.2** Entry Point & Configuration
- **5.3** Security (JWT + Spring Security)
- **5.4** Controllers (API Layer)
- **5.5** Service Layer
- **5.6** Scan Engine (Static Analysis)
- **5.7** Data Models (JPA Entities)
- **5.8** Repositories (Data Access)
- **5.9** Utilities

6. Frontend Deep Dive

- **6.1** Project Structure & Routing
- **6.2** Pages & Components
- **6.3** API Service & Hooks

7. GitHub Webhook Integration

8. Security Rules Reference (All 21 Rules)

9. Badge Gamification System

10. Email Notification System

11. Database Schema

12. API Endpoints Reference

13. Setup & Deployment Guide

14. Testing

15. Summary

1. Introduction & Project Overview

Secure Push Gateway is a full-stack web application that acts as a middleware between GitHub and development teams. It automatically intercepts code pushes via GitHub webhooks, performs static analysis on changed Java and JavaScript files, detects security vulnerabilities, persists results, and provides a rich dashboard experience with gamification badges and email notifications.

The system serves two types of users:

- **OWNER (Admin):** Bird's-eye view of all developers, all scans, pass/fail statistics, and charts
- **DEVELOPER:** Personal scan history, earned badges, clean push streaks, manual code scanning

Key Capabilities:

- Automated webhook-triggered scanning on every git push
- Manual code scanning via browser (paste code and scan instantly)
- 25+ security rules across Java and JavaScript
- JavaParser AST-based analysis for Java (deep structural code understanding)
- Regex-based pattern matching for JavaScript/TypeScript
- JWT stateless authentication with BCrypt password hashing
- Role-based access control (OWNER vs DEVELOPER)
- Badge gamification system (streak tracking, 5 badge types)
- HTML email notifications via Gmail SMTP with Thymeleaf templates
- In-app notification system with 30-second real-time polling
- HMAC-SHA256 webhook signature validation with constant-time comparison

2. System Architecture

2.1 High-Level 3-Tier Architecture

```
PRESENTATION TIER (Port 5173)
    React 18 + Vite + Tailwind CSS
    Pages: Login | Owner Dashboard | Developer Dashboard | Manual Scan | Scan Detail
    Axios HTTP Client + JWT Interceptor
                  |
                  | REST API (JSON over HTTP)
                  v
APPLICATION TIER (Port 8080)
    Spring Boot 3.3.0 + Spring Security
    Controllers: Auth, Scan, Developer, Badge, Notification, Webhook
    Services:    ScanOrchestration, Badge, Notification, Email
    Engine:      ScanEngineService (7 AST Visitors + 12 Regex Rules)
    Security:    JWT Filter, BCrypt, HMAC Validator
                  |
                  | JPA / Hibernate ORM
                  v
DATA TIER
    MySQL 8.0+
    Tables: users, scans, vulnerability_results, badges, notifications
```

2.2 Webhook Flow (Automated Scanning)

```
Developer --> git push --> GitHub
                           |
                           v
                POST /api/webhook/github  (HMAC-SHA256 validated)
                           |
                           v
                Returns HTTP 200 immediately (non-blocking)
                           |
                           v
                ScanOrchestrationService.processPushAsync() [@Async]
                           |
                +----------+-----------+
                |                      |
           Java files              JS/TS files
           (JavaParser AST)        (Regex engine)
           7 visitor rules         12 pattern rules
                |                      |
                +----------+-----------+
                           |
                           v
                MySQL: Scan + VulnerabilityResult persisted
                           |
                +----------+-----------+
                |                      |
           Email alerts          In-app notifications
           (Thymeleaf HTML)      (polled every 30s)
                           |
                Badge evaluation (streak logic)
                           |
                React Dashboard auto-updates
```

2.3 Manual Scan Flow

```
User pastes code in browser
        |
        v
Frontend POST /api/scans/manual  {fileName, code, repoName}
        |
        v
ScanController --> ScanEngineService.scan(fileName, code)
        |
        +-- Persists Scan + VulnerabilityResult records
        +-- Creates Notification
        +-- Evaluates Badges (if scan passes)
        +-- Returns JSON: {scanId, status, totalVulnerabilities, vulnerabilities[], newBadges[]}
```

2.4 Authentication Flow

```
POST /api/auth/login {username, password}
        |
        v
AuthController --> AuthenticationManager --> UserDetailsService --> DB lookup
        |
        v
BCrypt password verification
        |
        v
JwtUtil.generateToken(username, role) --> HMAC-SHA256 signed JWT
        |
        v
Response: {token, userId, username, role}
        |
        v
Frontend stores token in memory (module-scoped variable, NOT localStorage)
        |
        v
Axios interceptor adds "Authorization: Bearer <token>" to every request
        |
        v
JwtAuthFilter extracts token --> validates --> sets SecurityContext
```

3. Technology Stack & Software Used

3.1 Backend Technologies

Technology	Purpose
Java 21 (LTS)	Primary backend language with modern features (pattern matching, records, text blocks)
Spring Boot 3.3.0	Auto-configuration, embedded Tomcat, dependency injection, starter packages
Spring Security 6.x	JWT auth filter, BCrypt password encoder, stateless session, RBAC
Spring Data JPA	ORM with Hibernate, derived query methods, auto schema migration
JavaParser 3.25.8	Parses Java source into AST for deep structural analysis via Visitor pattern
JJWT 0.12.5	JWT generation/validation with HMAC-SHA256 signing, configurable expiration
MySQL Connector/J	JDBC driver for MySQL connectivity via Spring Data JPA
Lombok	Compile-time codegen: @Data, @Builder, @Slf4j eliminate boilerplate
Jackson	JSON serialization/deserialization, @JsonIgnore for output control
Spring Mail + Thymeleaf	Email sending with rich HTML templates via Gmail SMTP

3.2 Frontend Technologies

Technology	Purpose
React 18.3.1	Component-based UI with hooks (useState, useEffect, useContext, useCallback)
Vite 5.2.13	Next-gen build tool with lightning-fast HMR and ES module support
Tailwind CSS 3.4.4	Utility-first CSS framework for responsive design without custom CSS
Axios 1.7.2	HTTP client with request/response interceptors for JWT injection
React Router 6.23.1	Client-side routing with protected routes and role-based rendering
Recharts 2.12.7	React charting library for PieChart (pass/fail) and BarChart (developer scans)

3.3 Build Tools & Database

Tool	Purpose
MySQL 8.0+	Relational database with InnoDB engine, FK constraints, 5 tables
Maven 3.9+	Java build & dependency management, spring-boot-maven-plugin
npm + Vite	Node package manager + frontend dev server and production builds
Git	Version control, webhook integration trigger

4. Concepts & Design Patterns

4.1 Static Analysis

Static analysis examines source code without executing it. This project implements two approaches: **AST-based analysis (Java)** where JavaParser creates an Abstract Syntax Tree for deep structural inspection, and **regex-based analysis (JavaScript)** where regular expressions scan each line for known vulnerability patterns.

4.2 Abstract Syntax Tree (AST)

An AST is a tree representation where each node represents a code construct (classes, methods, variables, expressions). For example, `String password = "secret"` becomes a FieldDeclaration node containing a VariableDeclarator with a StringLiteralExpr initializer. The HardcodedSecretVisitor traverses this tree to find variables with secret-like names initialized with string literals.

4.3 Visitor Pattern

Separates algorithms from the objects they operate on. Each security rule is a separate Visitor class extending VoidVisitorAdapter. AST nodes accept visitors, and each visitor checks for specific vulnerability patterns. This makes rules modular - add new rules without modifying existing code.

4.4 JWT Authentication

JSON Web Tokens are self-contained tokens carrying user identity claims. The server creates a JWT with username + role, signs with HMAC-SHA256. The frontend stores the token in a JavaScript module variable (not localStorage - immune to XSS). Every API request includes the token in the Authorization header. The server validates without a database lookup.

4.5 HMAC-SHA256

Hash-based Message Authentication Code used for: (1) JWT signing to prevent token forgery, and (2) GitHub webhook validation where the server recomputes the HMAC and compares using constant-time comparison to prevent timing attacks.

4.6 Asynchronous Processing (@Async)

Webhook processing runs asynchronously. The controller returns HTTP 200 immediately, while the scan runs in a background thread from Spring's configurable thread pool (core=4, max=10). This prevents GitHub's 10-second webhook timeout.

4.7 Role-Based Access Control (RBAC)

Two roles: OWNER can view all developers, scans, and statistics. DEVELOPER can only view their own data. The frontend renders different dashboards, and the backend enforces access at the controller level.

4.8 Repository Pattern

Spring Data JPA repositories provide CRUD automatically. Custom queries use derived naming: `findByUsername(String)` becomes SELECT * FROM users WHERE username = ?.

4.9 Builder Pattern

All entities use Lombok's @Builder for fluent object construction: `Scan.builder().repoName("repo").status(FAIL).build()`.

4.10 Interceptor Pattern

The Axios client uses request interceptors to automatically attach JWT tokens to every outgoing API request.

5. Backend Deep Dive

5.1 Project Structure

The backend follows a layered architecture with clear separation of concerns:

```
backend/src/main/java/com/securepushgateway/
    SecurePushGatewayApplication.java     # Entry point (@EnableAsync)
    config/
        SecurityConfig.java               # JWT filter, CORS, Spring Security
        AppConfig.java                    # Beans: UserDetailsService, AuthManager
        DataSeeder.java                   # Seeds default admin on startup
    controller/
        AuthController.java               # Login, Register, Me
        ScanController.java               # Scan CRUD + Manual scan
        DeveloperController.java          # Developer profiles
        BadgeController.java              # Badge queries
        NotificationController.java       # Notification management
        WebhookController.java            # GitHub webhook handler
    engine/
        ScanEngineService.java            # Core scan (7 AST visitors + 12 regex rules)
    model/
        User.java, Scan.java, VulnerabilityResult.java, Badge.java, Notification.java
    repository/
        UserRepository, ScanRepository, VulnerabilityResultRepository,
        BadgeRepository, NotificationRepository
    service/
        ScanOrchestrationService.java     # Async webhook pipeline
        BadgeService.java                 # Streak + badge logic
        NotificationService.java          # In-app notifications
        EmailService.java                 # Thymeleaf email sending
    util/
        JwtUtil.java                      # JWT generation/validation
        HmacValidator.java                # GitHub HMAC-SHA256 validation
```

5.2 Entry Point & Configuration

SecurePushGatewayApplication.java

`@SpringBootApplication` combines @Configuration, @EnableAutoConfiguration, @ComponentScan. `@EnableAsync` enables Spring's async execution for the @Async scan pipeline.

SecurityConfig.java

- **Stateless sessions:** No server-side session; every request carries a JWT
- **JWT Filter:** Custom OncePerRequestFilter extracts and validates JWT from Authorization header
- **CORS:** Allows requests from http://localhost:5173 (frontend dev server)
- **CSRF disabled:** Stateless JWT auth doesn't need CSRF protection
- **Public endpoints:** /api/auth/** and /api/webhook/github (no auth required)
- **All other endpoints:** Require valid JWT

AppConfig.java

- **PasswordEncoder:** BCrypt encoder for password hashing
- **UserDetailsService:** Loads user from DB for Spring Security authentication
- **DaoAuthenticationProvider:** Wires UserDetailsService + PasswordEncoder
- **AuthenticationManager:** Used by AuthController for login
- **RestTemplate:** HTTP client for calling GitHub API

DataSeeder.java

Implements CommandLineRunner - runs on startup. Creates default OWNER account (admin/admin123) if not exists. Password is BCrypt hashed before storage.

5.3 Controllers (API Layer)

Controller	Base Path	Endpoints
AuthController	/api/auth/*	POST /login (auth + JWT), POST /register, GET /me
ScanController	/api/scans/*	GET / (list), GET /{id}, GET /stats, POST /manual
WebhookController	/api/webhook/*	POST /github (HMAC validated, async scan)
DeveloperController	/api/developers/*	GET / (OWNER only), GET /{id}
BadgeController	/api/badges/*	GET /{devId}, POST /evaluate/{devId}
NotificationController	/api/notifications/*	GET /{userId}, PUT /{id}/read

5.4 Service Layer

ScanOrchestrationService - Webhook Pipeline

The heart of the system. Runs @Async after webhook receipt:

- **Step 1:** Extracts pusher name, repo, branch, and commit SHAs from GitHub payload
- **Step 2:** Finds or auto-creates a User record for the GitHub pusher
- **Step 3:** For each commit, calls GitHub API to fetch changed file contents
- **Step 4:** Filters for .java, .js, .ts files only
- **Step 5:** Runs ScanEngineService.scan() on each file
- **Step 6:** Persists Scan + VulnerabilityResult records to MySQL
- **Step 7:** Creates in-app Notification for the developer
- **Step 8:** Sends email (developer + owner on FAIL)
- **Step 9:** Evaluates and awards Badges based on streak logic

BadgeService - Gamification

- **evaluate(developer, scan):** Checks if new badges should be awarded
- **getCurrentStreak(developer):** Counts consecutive PASS scans from most recent
- **isCleanWeek(developer):** Checks if all scans in current week passed

NotificationService & EmailService

- **NotificationService:** Creates SCAN_PASS, SCAN_FAIL, BADGE_AWARDED notifications
- **EmailService:** Sends HTML emails via Gmail SMTP using Thymeleaf templates

5.5 Scan Engine - ScanEngineService.java

The core static analysis engine dispatches to language-specific scanners based on file extension. Java files are parsed into an Abstract Syntax Tree using JavaParser, then 7 visitor classes traverse the tree. JavaScript/TypeScript files are scanned line-by-line with 12 regex patterns.

Java AST Visitors

Visitor Class	Rule ID	What It Detects
HardcodedSecretVisitor	JAVA-001	Checks FieldDeclaration and VariableDeclarationExpr for secret-named variables with strin
SqlInjectionVisitor	JAVA-002	Detects string concatenation in JDBC query arguments (executeQuery, execute, prepareS
UnsafeDeserializationVisitor	JAVA-003	Finds ObjectInputStream.readObject() calls (RCE risk)
DangerousMethodVisitor	JAVA-004	Detects Runtime.exec(), System.exit(), ProcessBuilder with concatenation
XssVectorVisitor	JAVA-005	Finds unescaped variables written to HTTP response writers
WeakCryptoVisitor	JAVA-006	Detects MD5, SHA-1, DES, RC4, RC2 algorithm strings
NullDereferenceVisitor	JAVA-007	Detects chained calls on get*/find* results without null check

If JavaParser fails to parse (e.g., code snippet without class wrapper), a regex fallback catches basic patterns for hardcoded secrets and SQL injection.

5.6 Data Models (JPA Entities)

Entity	Fields
User	id, username, email, password, role (OWNER/DEVELOPER), createdAt
Scan	id, developer (FK), repoName, commitSha, branch, status (PASS/FAIL/PENDING), totalVulnerabilities, scannedAt
VulnerabilityResult	id, scan (FK), ruleId, severity (CRITICAL/HIGH/MEDIUM/LOW), fileName, lineNumber, description, codeSnippet
Badge	id, developer (FK), badgeType (5 types), awardedAt, scan (FK)
Notification	id, user (FK), message, type (SCAN_PASS/SCAN_FAIL/BADGE_AWARDED), read, createdAt, scanId

Relationships:

- User 1:N Scan (one developer has many scans)
- Scan 1:N VulnerabilityResult (one scan has many findings)
- User 1:N Badge (one developer has many badges)
- User 1:N Notification

5.7 Utilities

JwtUtil

- generateToken(username, role): JWT with subject=username, claims={role}, expires=24h, HMAC-SHA256
- extractUsername(token): Parses JWT, returns subject
- validateToken(token, userDetails): Checks signature, expiration, username match

HmacValidator

- isValid(payload, signatureHeader): Validates GitHub X-Hub-Signature-256 header
- computeHmac(payload): HMAC-SHA256 of payload with shared secret
- timingSafeEquals(a, b): Constant-time comparison (prevents timing attacks)
- Dev mode: If secret is empty, returns true (skips validation)

6. Frontend Deep Dive

6.1 Project Structure & Routing

```
frontend/src/
    App.jsx                    # React Router with role-based routing
    main.jsx                   # React entry point
    index.css                  # Tailwind CSS imports
    services/api.js            # Axios client + JWT interceptor
    context/AuthContext.jsx     # Auth state (login/logout/user)
    hooks/useNotifications.js   # 30-second notification polling
    pages/
        Login.jsx              # Login + Register tabs
        OwnerDashboard.jsx     # Admin: stats, charts, all scans
        DeveloperDashboard.jsx # Developer: personal scans, badges
        ScanDetail.jsx         # Vulnerability detail view
        ManualScan.jsx         # Manual code submission
    components/
        Navbar.jsx             # Top nav with Scan Code button
        NotificationBell.jsx   # Notification dropdown
        ProtectedRoute.jsx     # Auth guard (redirect to /login)
```

Routes:

Path	Component	Access	Description
/login	Login.jsx	Public	Login/Register form
/dashboard	OwnerDashboard or DeveloperDashboard	Protected	Role-based dashboard
/scans/:scanId	ScanDetail.jsx	Protected	Vulnerability detail view
/scan	ManualScan.jsx	Protected	Manual code submission

6.2 Pages

Login.jsx

Tab-based login/register form with form validation and error handling. Default credentials shown: admin/admin123.

OwnerDashboard.jsx

- Stats cards: Total scans, pass rate, total developers, total vulnerabilities
- PieChart: PASS vs FAIL distribution (Recharts)
- BarChart: Scans per developer
- Recent scans table with color-coded status badges
- Developer list with streak information

DeveloperDashboard.jsx

- Recent scans table, earned badges with emoji icons, scan statistics

ScanDetail.jsx

- Scan metadata (repo, commit SHA, branch, timestamp), status badge (PASS/FAIL)
- Vulnerability list with severity coloring, rule IDs, line numbers, code snippets

ManualScan.jsx

- File name input + code textarea (monospace font)
- Sample buttons: Load Java Sample (with vulns), Load JS Sample (with vulns)
- Results: severity badges (CRITICAL/HIGH/MEDIUM/LOW), descriptions, code snippets

6.3 API Service & Hooks

api.js - Centralized HTTP Client

- Base URL: http://localhost:8080
- Request interceptor: Adds Authorization header with JWT token
- Response interceptor: Redirects to /login on 401 Unauthorized
- Exports: authApi, scansApi, developersApi, badgesApi, notificationsApi
- JWT stored in module-scoped variable (immune to XSS token theft)

useNotifications.js - Polling Hook

Polls GET /api/notifications/{userId} every 30 seconds. Returns notifications array, unread count, and markRead function. Auto-refreshes on mark-read, cleans up interval on unmount.

7. GitHub Webhook Integration

7.1 How It Works

When a developer pushes code to a GitHub repository, GitHub sends a POST request to /api/webhook/github containing the push event payload (pusher info, repository details, branch ref, array of commits). The WebhookController validates the HMAC-SHA256 signature, then ScanOrchestrationService processes each commit asynchronously - fetching file contents from GitHub API, running the scan engine, persisting results, and sending notifications.

7.2 HMAC Signature Validation

GitHub signs every webhook payload using HMAC-SHA256 with a shared secret. The server: (1) Receives the signature in X-Hub-Signature-256 header, (2) Recomputes the HMAC using the same secret and raw payload body, (3) Compares using constant-time comparison to prevent timing attacks, (4) Rejects the request if signatures don't match. In dev mode (empty secret), validation is skipped.

7.3 Setup Steps

- **Step 1:** Generate a GitHub Personal Access Token (Fine-grained, Contents: Read-only)
- **Step 2:** Set GITHUB_TOKEN and GITHUB_WEBHOOK_SECRET environment variables
- **Step 3:** Expose localhost using ngrok: ngrok http 8080
- **Step 4:** Add webhook in GitHub repo > Settings > Webhooks with the ngrok URL
- **Step 5:** Set Content type to application/json and select 'Just the push event'
- **Step 6:** Push code to the repo - scans run automatically

8. Security Rules Reference (All 21 Rules)

8.1 Java Rules (AST-based via JavaParser)

Rule ID	Severity	Name	Detection	Remediation
JAVA-001	CRITICAL	Hardcoded Secrets	String vars named password/apiKey/secret/token with string literals	Use environment variables or secrets manager
JAVA-002	CRITICAL	SQL Injection	String concatenation in JDBC query args (executeQuery, executeUpdate, prepareStatement)	Use PreparedStatement with ? params
JAVA-003	HIGH	Unsafe Deserialization	ObjectInputStream.readObject() calls	Use filtering OIS or JSON serialization
JAVA-004a	CRITICAL	Command Injection	Runtime.getRuntime().exec() calls	Use ProcessBuilder with arg list
JAVA-004b	MEDIUM	System.exit()	Abrupt JVM termination bypasses security handlers	Throw exceptions instead
JAVA-004c	HIGH	ProcessBuilder Injection	ProcessBuilder with string concatenation	Pass args as separate list elements
JAVA-005	HIGH	XSS (Response Writer)	Unescaped vars written to HTTP response writer	Use OWASP Java Encoder
JAVA-006	HIGH	Weak Cryptography	MD5, SHA-1, DES, RC4, RC2 algorithm strings	Use SHA-256+, AES, BCrypt
JAVA-007	MEDIUM	Null Dereference	Chained calls on get*/find* without null check	Use Optional or null guard

8.2 JavaScript Rules (Regex-based)

Rule ID	Severity	Name	Pattern Detected	Fix
JS-001	CRITICAL	Hardcoded Secrets	password/apiKey/token/secret = "..."	Use process.env
JS-002	HIGH	eval()	eval(...) usage	Use JSON.parse() or Function()
JS-003	HIGH	innerHTML XSS	.innerHTML = <variable>	Use textContent or DOMPurify
JS-004	HIGH	document.write	document.write(<variable>)	Use DOM manipulation
JS-005	CRITICAL	SQL Template Literal	query(`SELECT...${var}`)	Use parameterized queries
JS-006	MEDIUM	Console Log Secrets	console.log(...password|token...)	Remove before production
JS-007	HIGH	Prototype Pollution	__proto__ = or Object.assign({},...)	Validate property names
JS-008	LOW	Insecure Random	Math.random()	Use crypto.getRandomValues()
JS-009	MEDIUM	dangerouslySetInnerHTML	React dangerouslySetInnerHTML	Sanitize with DOMPurify
JS-010	MEDIUM	Open Redirect	location.href = <variable>	Validate URL whitelist
JS-011	LOW	Hardcoded Localhost	http://localhost or 127.0.0.1	Use env variables
JS-012	MEDIUM	setTimeout String	setTimeout("...", ...)	Pass function reference

9. Badge Gamification System

The badge system incentivizes developers to write secure code through progressive achievement milestones:

Badge Type	Requirement	Description
FIRST_CLEAN_PUSH	1 clean scan	First ever vulnerability-free commit
STREAK_3	3 consecutive PASS	Three clean pushes in a row
STREAK_5	5 consecutive PASS	Five clean pushes in a row
STREAK_10	10 consecutive PASS	Security champion - ten clean pushes!
CLEAN_WEEK	All scans in a week PASS	Entire calendar week vulnerability-free

Streak Calculation Logic:

- Fetch all scans for the developer, ordered by date (newest first)
- Count consecutive PASS scans from the most recent
- Stop counting at the first FAIL scan (streak resets to 0)
- Compare streak count against badge thresholds
- Only award badges not already earned (no duplicates)

10. Email Notification System

The system sends rich HTML emails via Gmail SMTP using Thymeleaf templates:

Template	Recipient	Content
scan-pass.html	Developer	Congratulates on clean push with repo, branch, commit info
scan-fail.html	Developer	Alerts about vulnerabilities with severity, rule ID, file, line, description
badge-awarded.html	Developer	Notifies about newly earned badge
owner-alert.html	Owner	Alerts when any developer's scan fails (developer name, repo, vuln count)

Configuration:

- Gmail SMTP: smtp.gmail.com:587, TLS enabled
- Requires Gmail App Password (not regular password) - enable 2FA first
- All email sending is @Async to avoid blocking the scan pipeline
- Gracefully handles failures (logs error, doesn't crash the scan)

11. Database Schema

Five tables with foreign key relationships. All tables are auto-created by Hibernate (ddl-auto=update).

users

Column	Type	Constraints
id	BIGINT	PK, AUTO_INCREMENT
username	VARCHAR	UNIQUE, NOT NULL
email	VARCHAR	UNIQUE, NOT NULL
password	VARCHAR	NOT NULL (BCrypt hash)
role	ENUM	'OWNER' or 'DEVELOPER'
created_at	DATETIME	Auto-set on create

scans

Column	Type	Constraints
id	BIGINT	PK, AUTO_INCREMENT
developer_id	BIGINT	FK -> users.id, NOT NULL
repo_name	VARCHAR	NOT NULL
commit_sha	VARCHAR	NOT NULL
branch	VARCHAR	
status	ENUM	'PASS', 'FAIL', or 'PENDING'
total_vulnerabilities	INT	
scanned_at	DATETIME	Auto-set on create

vulnerability_results

Column	Type	Constraints
id	BIGINT	PK, AUTO_INCREMENT
scan_id	BIGINT	FK -> scans.id, NOT NULL
rule_id	VARCHAR	NOT NULL (e.g., 'JAVA-001')
severity	ENUM	'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'
file_name	VARCHAR	NOT NULL

Column	Type	Constraints
line_number	INT	
description	VARCHAR(1000)	NOT NULL
code_snippet	VARCHAR(2000)	

badges

Column	Type	Constraints
id	BIGINT	PK, AUTO_INCREMENT
developer_id	BIGINT	FK -> users.id
badge_type	ENUM	5 badge types
awarded_at	DATETIME	Auto-set
scan_id	BIGINT	FK -> scans.id

notifications

Column	Type	Constraints
id	BIGINT	PK, AUTO_INCREMENT
user_id	BIGINT	FK -> users.id
message	VARCHAR	NOT NULL
type	ENUM	'SCAN_PASS', 'SCAN_FAIL', 'BADGE_AWARDED'
read	BOOLEAN	DEFAULT FALSE
created_at	DATETIME	Auto-set
scan_id	BIGINT	For navigation

Entity Relationships:

- User 1:N Scan (one developer has many scans)
- Scan 1:N VulnerabilityResult (one scan has many findings)
- User 1:N Badge (one developer has many badges)
- User 1:N Notification
- Scan 1:N Badge (badge linked to triggering scan)

12. API Endpoints Reference

All endpoints return JSON. Authentication via 'Authorization: Bearer <JWT>' header.

Method	Endpoint	Auth	Description
POST	/api/auth/login	No	Login, returns JWT token + user info
POST	/api/auth/register	No	Register new user (default: DEVELOPER)
GET	/api/auth/me	Yes	Get current authenticated user
GET	/api/scans	Yes	List scans (OWNER=all, DEV=own)
GET	/api/scans/{id}	Yes	Scan detail with vulnerability list
GET	/api/scans/developer/{id}	Yes	Specific developer's scans
GET	/api/scans/stats	OWNER	Dashboard statistics
POST	/api/scans/manual	Yes	Submit code for manual scanning
POST	/api/webhook/github	HMAC	GitHub push event receiver
GET	/api/developers	OWNER	List all developers
GET	/api/developers/{id}	Yes	Developer profile with stats
GET	/api/badges/{devId}	Yes	Developer's earned badges
POST	/api/badges/evaluate/{devId}	Yes	Trigger badge evaluation
GET	/api/notifications/{userId}	Yes	User's notifications
PUT	/api/notifications/{id}/read	Yes	Mark notification as read

Sample Request/Response

POST /api/auth/login

```
Request:  {"username": "admin", "password": "admin123"}
Response: {"token": "eyJhbGciOi...", "userId": 1, "username": "admin", "role": "OWNER"}
```

POST /api/scans/manual

```
Request:  {"fileName": "Test.java", "code": "public class Test {...}", "repoName": "manual-test"}
Response: {
  "scanId": 5,
  "status": "FAIL",
  "totalVulnerabilities": 2,
  "vulnerabilities": [
    {"ruleId": "JAVA-001", "severity": "CRITICAL", "lineNumber": 2, "description": "...", "codeSnippet": "..."
  ],
  "newBadges": []
}
```

13. Setup & Deployment Guide

13.1 Prerequisites

Tool	Version	Check Command
Java JDK	21+	java -version
Maven	3.9+	mvn -version
Node.js	18+ (v22 LTS recommended)	node -v
npm	9+	npm -v
MySQL	8.0+	mysql --version
Git	2.x	git --version

13.2 Step-by-Step Setup

Step 1: Clone the repository

```
git clone https://github.com/YOUR_USERNAME/secure-push-gateway.git
cd secure-push-gateway
```

Step 2: Create MySQL database

```
mysql -u root -p
CREATE DATABASE IF NOT EXISTS securepushgateway;
```

Step 3: Set environment variables

```
# Required
export DB_URL="jdbc:mysql://localhost:3306/securepushgateway?createDatabaseIfNotExist=true&useSSL=false&server
export DB_USERNAME="root"
export DB_PASSWORD="your_mysql_password"
export JWT_SECRET="your-256-bit-secret-key"

# Optional (GitHub webhook integration)
export GITHUB_TOKEN="ghp_your_token"
export GITHUB_WEBHOOK_SECRET="your-webhook-secret"

# Optional (Email notifications)
export MAIL_USERNAME="your-email@gmail.com"
export MAIL_PASSWORD="your-gmail-app-password"
```

Step 4: Start the backend

```
cd backend
mvn spring-boot:run
# Runs on http://localhost:8080
```

Step 5: Start the frontend

```
cd frontend
npm install
npm run dev
# Runs on http://localhost:5173
```

Step 6: Open the application

- Go to http://localhost:5173
- Login with admin / admin123 (default Owner account)
- Or Register to create a new Developer account
- Click 'Scan Code' to test the manual scanner

13.3 Optional: GitHub Webhook Setup

- **1.** Generate GitHub Personal Access Token (Contents: Read-only)
- **2.** Set GITHUB_TOKEN and GITHUB_WEBHOOK_SECRET env vars
- **3.** Install and run ngrok: `ngrok http 8080`
- **4.** Add webhook in GitHub repo Settings with ngrok URL
- **5.** Push code - scans run automatically

13.4 Troubleshooting

Problem	Solution
MySQL Connection refused	Ensure MySQL is running: sudo systemctl start mysql
Access denied for MySQL	Verify DB_USERNAME and DB_PASSWORD
Frontend 'Network error'	Backend must be running on port 8080
Login fails with admin/admin123	Check backend logs for 'Default OWNER account created'
Webhook returns 401	GITHUB_WEBHOOK_SECRET must match in app and GitHub
Scans always PASS on webhook	Set GITHUB_TOKEN to fetch file contents from API
Email not sending	Use Gmail App Password (enable 2FA first)

14. Testing

14.1 Automated Test Scripts

test-e2e.sh - 44 API Tests

Comprehensive end-to-end API test suite covering:

Category	Tests
Authentication	Login, register, invalid credentials, JWT validation
Webhook	Push events, HMAC signature validation
Scan CRUD	List scans, scan detail, developer scans, stats
RBAC	Developer cannot access Owner-only endpoints
Notifications	Creation after scan, mark as read
Badges	Evaluation, retrieval
Security	Unauthenticated access blocked (401/403)
CORS	Correct headers returned

test-manual-scan.sh - Manual Scan Tests

Test Case	Expected Result
1. Java Vulnerable Code	Expects FAIL, 4+ vulns, JAVA-001/002/004a/006 detected
2. JavaScript Vulnerable Code	Expects FAIL, 5+ vulns, JS-001/002/003/005/006/012 detected
3. Clean Java Code	Expects PASS, 0 vulnerabilities
4. Notifications	Verifies notifications created for all scans
5. Unsupported File (.py)	Expects PASS, 0 vulns (no scanner for Python)
6. Auth Required	Unauthenticated request returns 401/403

Unit Tests - ScanEngineServiceTest.java

JUnit tests for all Java and JavaScript scan rules. Run with:

```
cd backend
mvn test
```

14.2 Running Tests

```
# API E2E tests (backend must be running on port 8080)
bash test-e2e.sh

# Manual scan tests (backend must be running)
```

```
bash test-manual-scan.sh

# Unit tests
cd backend && mvn test
```

15. Summary

Secure Push Gateway demonstrates a complete DevSecOps pipeline from code push to vulnerability detection, notification, and gamification. The system provides a practical implementation of automated security scanning that can be integrated into real development workflows.

The Complete Pipeline:

- **1.** Code is pushed to GitHub
- **2.** A webhook triggers automated security scanning
- **3.** The scan engine analyzes code using AST (Java) and regex (JavaScript)
- **4.** 25+ rules detect real-world vulnerabilities
- **5.** Results are persisted and displayed on role-based dashboards
- **6.** Email and in-app notifications keep developers informed
- **7.** A gamification badge system rewards secure coding practices
- **8.** Manual scanning allows testing without GitHub integration

Technical Skills Demonstrated:

Skill Area	Technologies / Concepts
Full-Stack Development	Spring Boot 3 backend + React 18 frontend
Security Engineering	JWT auth, HMAC validation, static analysis, vulnerability detection
API Design	RESTful endpoints with proper authentication and RBAC
Database Design	Normalized schema with proper FK relationships
Async Processing	Non-blocking webhook pipeline with @Async thread pool
Design Patterns	Visitor, Builder, Repository, Interceptor, Observer
Modern Tooling	Vite, Tailwind CSS, Recharts, JavaParser, Lombok
Testing	E2E API tests, manual scan tests, JUnit unit tests
DevSecOps	GitHub webhook integration, automated scanning, badge gamification

--- End of Documentation ---